



FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA
Graduação ON em Inteligência Artificial

Arthur Prudêncio Soares - RM569295
Caroline Coelho Mendes - RM570370
Leandro Paiva - RM572159
Lucas Viana de Lima - RM571835
Matheus Tavares Lima - RM572808



**ORBITHAM: ESTAÇÃO TERRENA PARA RASTREAMENTO DE
SATÉLITES**

Global Solution 2026.1

QUERO CONCORRER

São Paulo
2026

Arthur Prudêncio Soares - RM569295
Caroline Coelho Mendes - RM570370
Leandro Paiva - RM572159
Lucas Viana de Lima - RM571835
Matheus Tavares Lima - RM572808

ORBITHAM: ESTAÇÃO TERRENA PARA RASTREAMENTO DE SATÉLITES

Trabalho apresentado à Faculdade de Informática e
Administração Paulista (FIAP) como requisito da
Global Solution 2026.1 do curso de Graduação ON
em Inteligência Artificial.

São Paulo
2026

1 INTRODUÇÃO

O espaço deixou de ser apenas exploração científica e se tornou um dos maiores ecossistemas de inovação e transformação econômica do planeta. Satélites monitoram o clima, conectam regiões remotas e apoiam decisões em escala global. Nesse cenário, a comunicação via satélite por radioamadores depende de operar passagens de satélites de órbita baixa (LEO): janelas curtas em que o satélite fica acima do horizonte da estação.

Operar uma passagem exige responder, em tempo real, a quatro perguntas: quando o satélite aparece, por quanto tempo, com que qualidade (elevação) e para onde apontar a antena. Atualmente essas informações ficam espalhadas em ferramentas diferentes.

O OrbitHam, resposta do grupo à Global Solution 2026.1, reúne tudo isso em uma solução única e prática, com três frentes que compartilham o mesmo motor de propagação orbital (Skyfield/SGP4): uma aplicação web, uma camada de análise de dados em Python e a automação de um rotor de antena com ESP32.

2 DESENVOLVIMENTO

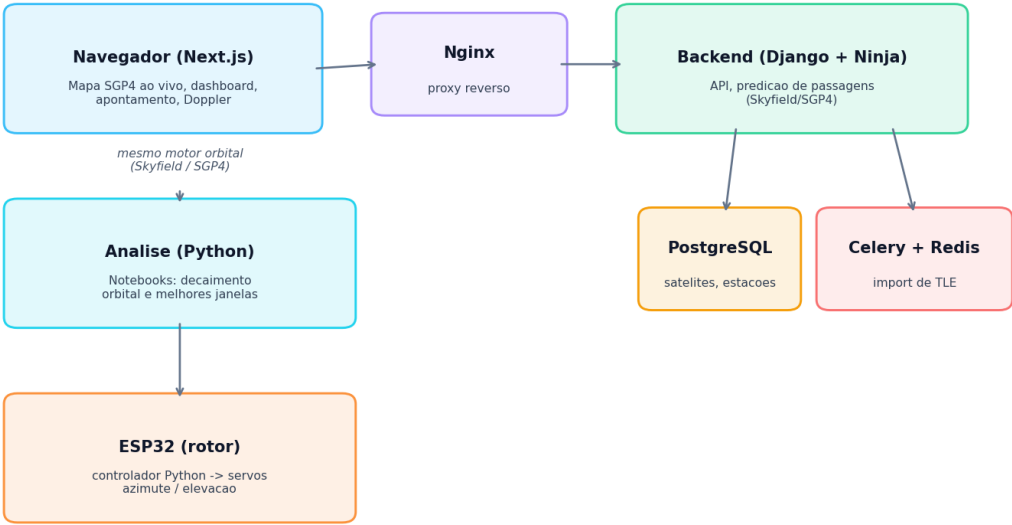
2.1 Visão geral da solução

- a) Aplicação web: mapa orbital ao vivo (SGP4 no navegador), previsão de passagens, dashboard com contagem regressiva e apontamento de antena com azimuth, elevação e correção Doppler;
- b) Análise de dados (Python/Jupyter): previsão de decaimento orbital e análise de melhores janelas de operação, com Pandas, Matplotlib/Seaborn e aprendizado de máquina introdutório (scikit-learn);
- c) Automação (ESP32): um controlador em Python calcula o apontamento e envia por Wi-Fi para um ESP32 que move dois servos, apontando a antena fisicamente para o satélite.

2.2 Arquitetura

O backend segue as camadas API, Service, Repository e Model: as rotas nunca acessam o banco diretamente e as entradas e saídas passam por esquemas. A propagação orbital roda em Python no backend (Skyfield) e no navegador (satellite.js), permitindo o tempo real sem custo de servidor. O mesmo cálculo alimenta a análise de dados e o rotor, mantendo as três frentes coerentes, como ilustra a figura a seguir.

Figura 1 - Arquitetura da solução OrbitHam.
OrbitHam - Arquitetura da solucao



Fonte: elaborado pelos autores (2026).

2.3 Códigos principais

A predição de passagens no backend usa o Skyfield (SGP4):

```
times, events = satellite.find_events(
    observer, t0, t1, altitude_degrees=10.0)
for ti, event in zip(times, events):
    if event == 0: # subida (rise)
        current = {'rise': ti.utc_datetime()}
    elif event == 1: # pico (culminacao)
        alt, _, _ = (satellite - observer).at(ti).altaz()
        current['max_elevation'] = alt.degrees
    elif event == 2: # descida (set)
        passes.append(current)
```

Quadro - predição de passagens (backend).

O apontamento (azimute e elevação) e a distância ao satélite:

```
alt, az, distance = (satellite - observer).at(t).altaz()
azimute = az.degrees # para onde apontar a antena
elevacao = alt.degrees # altura acima do horizonte
visivel = elevacao >= 0 # satellite acima do horizonte
```

Quadro - cálculo de apontamento (controlador do rotor).

A regressão do decaimento orbital, na análise de dados:

```

coeffs = np.polyfit(daily['day'], daily['altitude_km'], 2)
poly = np.poly1d(coeffs)
for day in range(6000):      # ate o limiar de reentrada
    if poly(day) <= 120.0:
        return day          # estimativa de reentrada

```

Quadro - regressão do decaimento orbital.

O firmware do ESP32 recebe os ângulos e move os servos:

```

void handlePoint() {
    float az = server.arg("az").toFloat();
    float el = server.arg("el").toFloat();
    azServo.write(map(az, 0, 360, 0, 180));
    elServo.write((int)(el * 2));    // 0..90 -> 0..180
}

```

Quadro - firmware do rotor (ESP32).

2.4 Decisões do grupo

- Propagação SGP4 no navegador para o mapa em tempo real, evitando custo de servidor e latência;
- Skyfield/SGP4 em Python no backend e na análise: preciso, testável e executável offline;
- Análise em notebooks reproduzíveis, para evidenciar o passo a passo de ciência de dados;
- Mesmos limiares de qualidade de passagem (45 e 25 graus) no app e nos notebooks, mantendo produto e análise consistentes;
- Comunicação app/ESP32 por HTTP simples, sem bibliotecas extras no firmware.

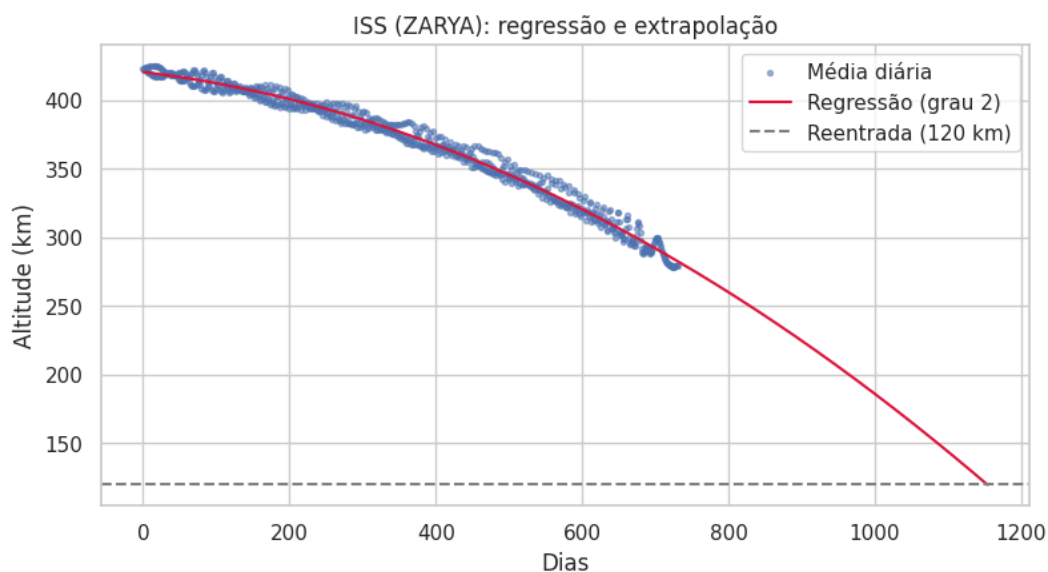
3 RESULTADOS ESPERADOS

A solução entrega um produto utilizável e uma análise que orienta a operação. A seguir, os principais resultados.

3.1 Previsão de decaimento orbital

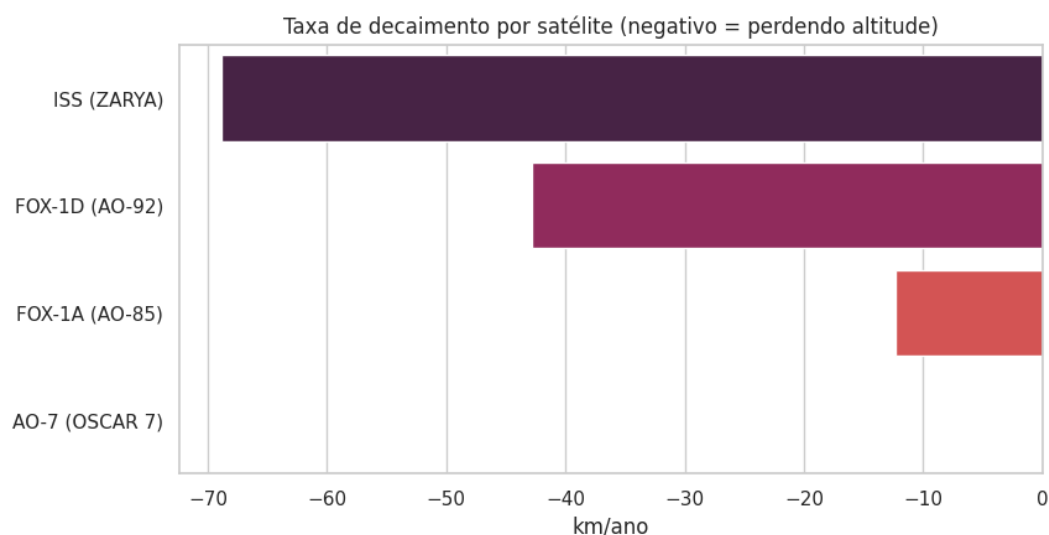
Propagando o TLE da Estação Espacial Internacional e ajustando uma regressão, o modelo estima perda de altitude de cerca de 69 km por ano e reentrada por volta de 2027. A comparação entre satélites mostra que objetos baixos decaem rápido, enquanto os mais altos são quase estáveis.

Figura 2 - Regressão do decaimento da ISS e limiar de reentrada.



Fonte: elaborado pelos autores (2026).

Figura 3 - Taxa de decaimento comparada entre satélites.

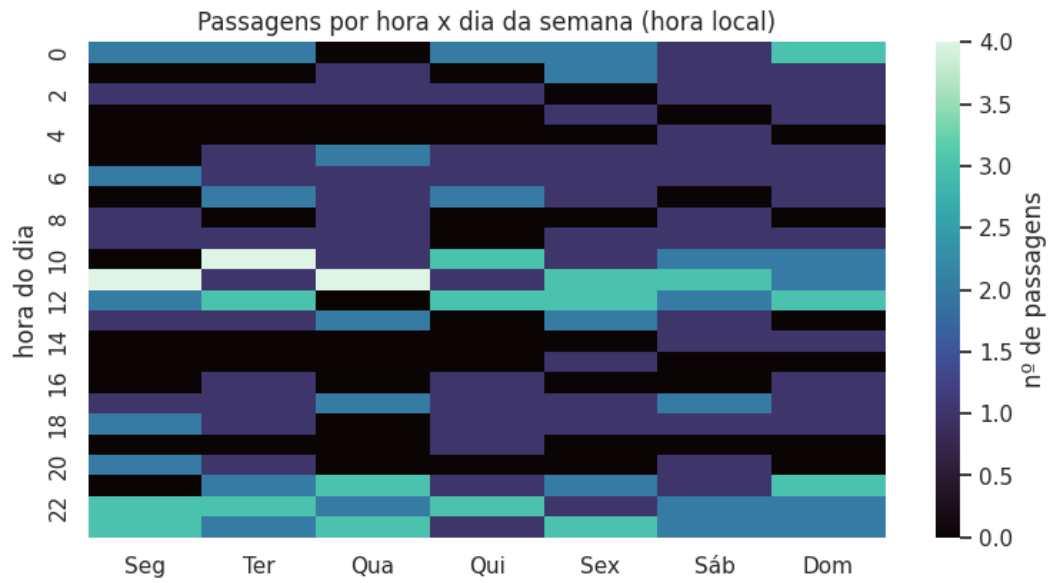


Fonte: elaborado pelos autores (2026).

3.2 Análise de passagens e melhores janelas

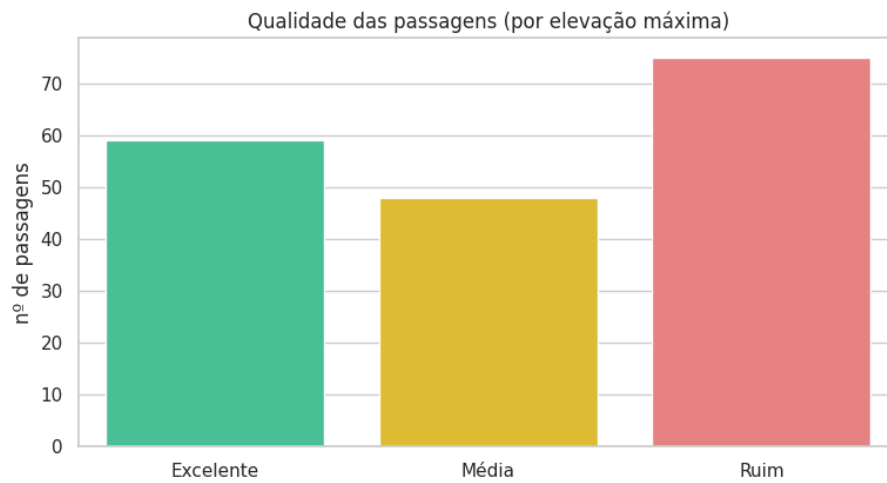
Para 182 passagens em 14 dias sobre São Paulo, o mapa de calor revela os melhores horários de operação (em torno das 11h e 22h). Um classificador prevê a qualidade da passagem com cerca de 84% de acurácia, sendo a duração a variável mais informativa.

Figura 4 - Passagens por hora e dia da semana (melhores janelas).



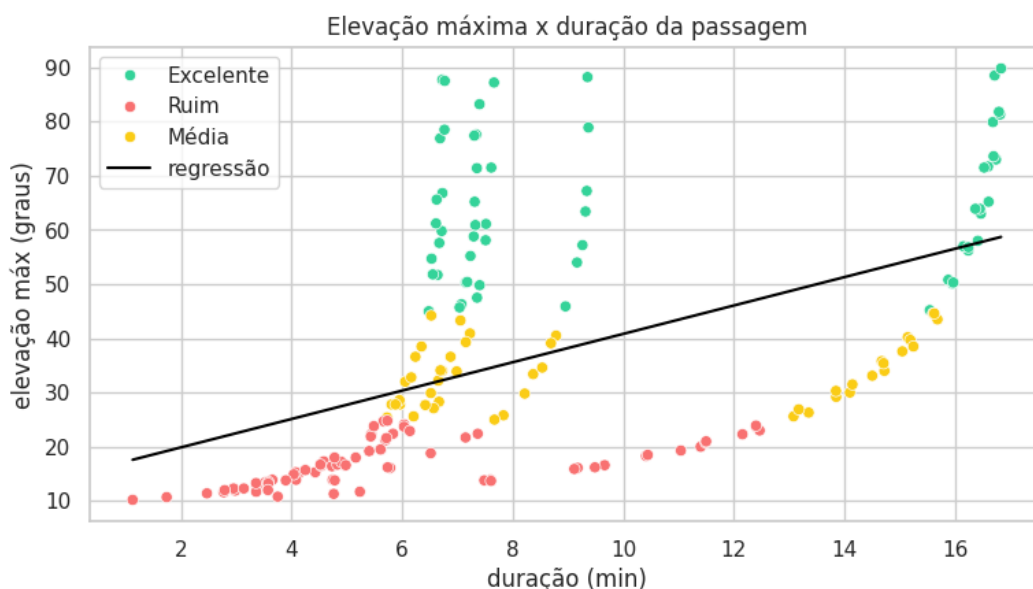
Fonte: elaborado pelos autores (2026).

Figura 5 - Distribuição de qualidade das passagens.



Fonte: elaborado pelos autores (2026).

Figura 6 - Relação entre duração e elevação máxima (aprendizado de máquina).



Fonte: elaborado pelos autores (2026).

4 CONCLUSÕES

O OrbitHam demonstra, na prática, como a tecnologia espacial melhora processos na Terra: transforma dados orbitais públicos (TLE) em decisões operacionais claras, de quando e para onde apontar a antena até a previsão de decaimento de um satélite.

A solução integra as competências do curso: programação em Python e TypeScript, análise de dados e aprendizado de máquina introdutório, visualização e automação com ESP32, tudo amarrado pelo mesmo motor orbital. As três frentes se reforçam: o app opera, a análise orienta e o hardware executa o apontamento.

Como evolução, vislumbramos realimentação de posição por sensor inercial no rotor, integração com rádio definido por software (SDR) para recepção real, aplicativo móvel e realimentação da análise com o histórico de TLE já armazenado pelo backend.

REFERÊNCIAS

RHODES, Brandon. Skyfield: high precision research-grade positions for planets and Earth satellites. Disponível em: <https://rhodesmill.org/skyfield/>. Acesso em: jun. 2026.

VALLADO, David A. et al. Revisiting Spacetrack Report #3 (SGP4). AIAA, 2006.

CELESTRAK. NORAD GP Element Sets (TLE). Disponível em: <https://celestrak.org/>. Acesso em: jun. 2026.

SATELLITE.JS. SGP4/SDP4 implementation in JavaScript. Disponível em: <https://github.com/shashwatak/satellite-js>. Acesso em: jun. 2026.

FIAP. Global Solution 2026.1: economia espacial. São Paulo: FIAP, 2026.